

Knowledge Graphs, the Missing Link in Agentic AI-based Formal Verification

Vaisakh Naduvodi Viswambharan¹, Keerthan Koppam Radhakrishna¹, Deepak Narayan Gadde¹, Aman Kumar²

¹Infineon Technologies Dresden AG & Co. KG, Germany

²Infineon Technologies Semiconductor India Private Limited, India

Abstract—Recent advances in Large Language Models (LLMs) have enabled workflows that generate SystemVerilog Assertions (SVAs) from natural-language specifications, with the potential to accelerate Formal Verification (FV). However, high-quality assertion synthesis remains challenging because specifications are often ambiguous or incomplete and critical micro-architectural details reside in the Register Transfer Level (RTL). Many existing approaches treat the specification and RTL as loosely structured text, which weakens specification-to-RTL grounding and leads to semantic mismatches and frequent syntax failures during formal parsing and elaboration. This work addresses these limitations with a verification-centric Knowledge Graph (KG) constructed from structured Intermediate Representations (IRs) extracted from the specification, RTL, and formal-tool feedback, including syntax diagnostics, Counterexamples (CEXs), and coverage reports. The KG links requirements, design hierarchy, signals, assumptions, and properties to provide traceable, design-grounded context for generation. A multi-agent workflow queries and updates this KG to generate SVAs and to drive three refinement loops: syntax repair guided by tool diagnostics, CEX-guided correction using trace links, and coverage-directed property augmentation. Evaluation across seven benchmark designs indicates that KG-based context retrieval improves specification-to-RTL grounding and consistently produces compilable SVAs with low syntax-repair overhead. The approach achieves formal coverage ranging from 78.5% to 99.4%, though convergence exhibits design dependence with complex temporal and arithmetic reasoning remaining challenging for current LLM capabilities.

Index Terms—Formal Verification, Knowledge Graphs, Agentic AI, LLMs

I. INTRODUCTION

FV offers strong correctness for RTL designs by proving that key behaviors hold across all reachable states, including corner cases that are difficult to cover with simulation. Central to FV is using SVAs to encode intended behavior as formal properties. However, writing accurate assertions requires deep understanding of both the specification and RTL implementation, making assertions time-consuming and error-prone.

Advances in LLMs enable automated synthesis of SVAs from natural-language specifications [1], [2], [3], [4], [5]. To produce semantically correct SVAs, models need context linking to the relevant RTL hierarchy, and signal interactions [6]. Accordingly, prior work builds context through structured specification

Under grant 101194371, Rigoletto is supported by Chips Joint Undertaking and its members, including top-up funding by the National Funding Authorities from involved countries. Rigoletto is also funded by the Federal Ministry of Research, Technology and Space under the funding code 16MEE0548S. The responsibility for the content of this publication lies with the author.

processing, staged prompting [2], [7] and RTL-derived prompt enrichment [8], [9]. Common strategies include schema-guided normalization [3] and retrieval-augmented context selection [10], [11]. However, studies report that these assertions show syntax and semantic errors requiring iterative correction and manual triage [3], [12], motivating representations with explicit, persistent links across requirements, RTL semantics, generated properties, and verification outcomes.

KG-based structuring offers more explicit grounding. AssertionForge [6] builds a KG linking specification artifacts to the RTL hierarchy and signals for structured context. Yet, formal-tool outcomes, including CEXs, and coverage gaps, are typically not represented as queryable KG content, limiting traceability and hindering systematic closed-loop refinement.

To address these limitations, this work proposes a verification-centric methodology that builds a KG from structured IRs extracted from the specification, RTL, and formal-tool feedback. LLM agents query and update the KG to support design-grounded property generation and three refinement loops: syntax repair guided by parser and elaboration diagnostics, CEX-guided correction using traces, and coverage-directed augmentation from tool-reported gaps.

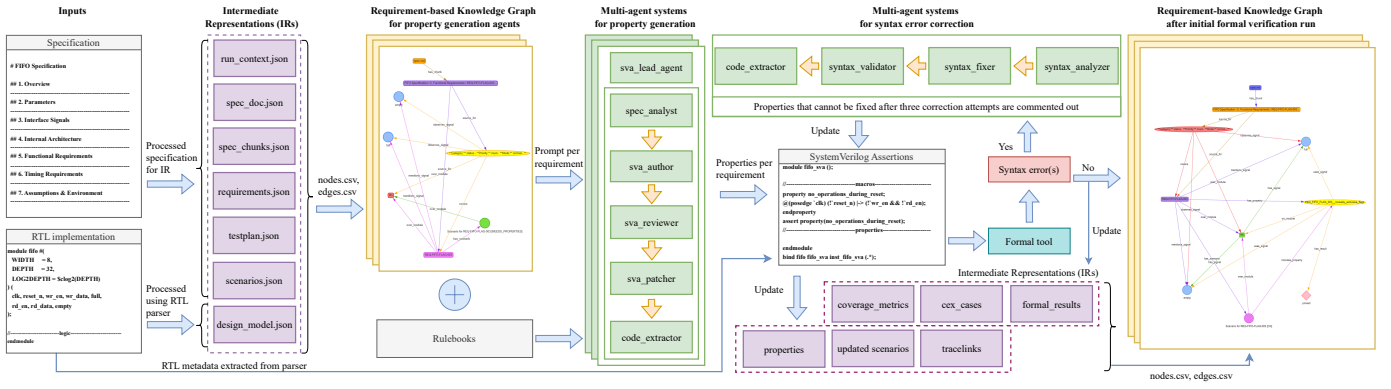
The main contributions of this paper are as follows:

- A verification-centric KG construction based on structured IRs, integrating specification intent, RTL structure, generated properties, and formal-tool results.
- A multi-agent workflow that uses KG-grounded context to generate and refine SVAs with traceability across requirements, design elements, properties, and results.
- An evaluation on open-source designs quantifying the impact of KG-grounded context on property generation effectiveness, syntactic validity, and CEX refinement.

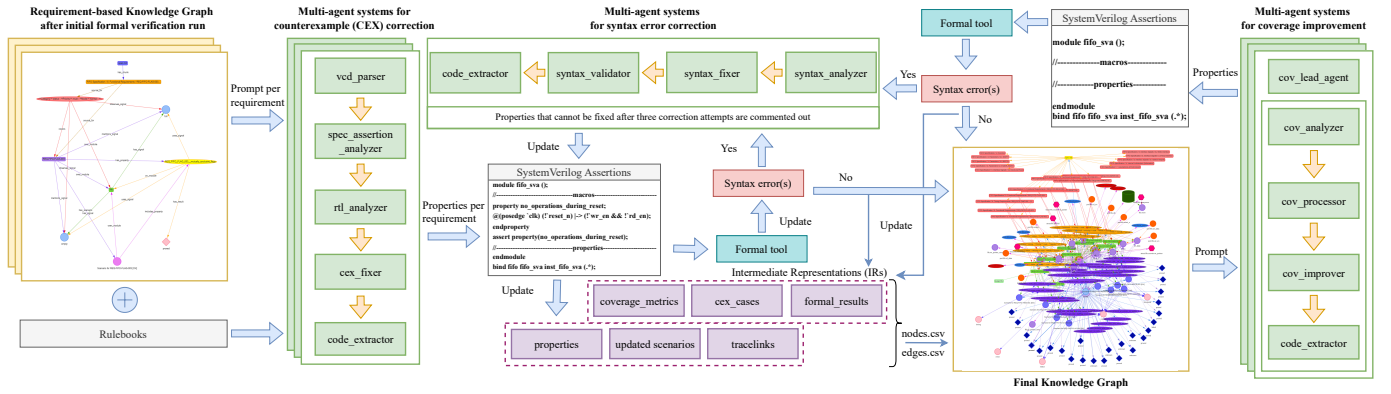
II. RELATED WORK

In LLM-assisted SVA generation, prior works differ in representing specification intent and design context; key challenge is grounding requirements and RTL semantics, while enabling iterative refinement from formal verification outcomes.

Specification-driven approaches synthesize SVAs from natural-language specifications, but are limited by ambiguous and missing behavioral constraints [1], [2]. To improve reliability, many systems structure specifications before generation: ChIRAAG [3] uses JSON schema-guided representations,



(a) Knowledge Graphs and multi-agent systems used for property generation and syntax error correction



(b) Knowledge Graphs and multi-agent systems used for CEX correction, coverage improvement and syntax error correction

Fig. 1. End-to-end workflow of the proposed approach. NetworkX is used for KG construction, and PyVis provides interactive HTML visualization.

AssertLLM [2] decomposes extraction and synthesis while Spec2Assertion [7] improves functional correctness via progressive regularization and structured reasoning yet remaining largely pre-RTL. These approaches improve consistency but struggle when micro-architectural details are absent.

In contrast, RTL-driven approaches use static analysis of implementation artifacts to guide assertion generation, improving signal relevance but risking intent drift without explicit code links [8], [9]. Hybrid approaches connect intent and implementation more explicitly: AssertionForge [6] constructs a KG linking specification artifacts to the RTL hierarchy and signals for structured context. Retrieval augmentation and structured prompting further focus models on relevant context: Retrieval Augmented Generation (RAG)-based approaches retrieve targeted fragments to reduce hallucinations [10], and LISA [11] combines retrieval with structured reasoning to improve temporal correctness and reduce retries. While effective, these methods rely on complete cross-artifact links between requirements, signals, and behaviors.

Despite these advances, studies still report recurring syntax errors and semantic mismatches requiring iterative fixes [12]. Some systems add limited feedback loops, such as simulation- or test-driven repair prompting [3], [13]. Saarthi [4] advances with an agentic, end-to-end formal workflow coordinating ver-

ification planning, assertion generation, formal proving, and coverage evaluation, yet formal-tool outputs remain transient rather than persistent, queryable artifacts.

Overall, prior work improves SVA generation via structured prompting, retrieval, and agentic decomposition but traceability remains weak without structured, queryable tool outputs. Thus, this work introduces a verification-centric KG constructed from structured IRs spanning specification intent, RTL structure, generated properties, and formal-tool feedback. This foundation enables multi-agent generation and closed-loop refinement driven by syntax diagnostics, CEXs, and coverage gaps, with traceability from requirements to assertions and formal outcomes.

III. METHODOLOGY

The workflow is organized into four layers:

- (i) Inputs: specification, RTL, configuration, and agents
- (ii) Typed JSON IRs: normalize requirements, RTL structure, properties, and verification outcomes
- (iii) Runtime KGs: retrieves localized context neighborhoods bounded by task relevance and provides traceable, design-grounded context
- (iv) Agent orchestration: routes property generation, syntax/CEX repair, and coverage improvement loops

TABLE I
Intermediate Representation artifacts and their roles in the verification workflow

IR Artifact	Purpose
<i>spec_chunks.json</i>	Hierarchical specification decomposition with heading structure, semantic tags, enabling targeted context retrieval
<i>requirements.json</i>	Extracted requirements with identifiers, natural language text, categories, priorities, traceability to specification chunks
<i>testplan.json</i>	Per-requirement verification plans, observable signals, stimulus-response behaviors, timing constraints
<i>design_model.json</i>	RTL metadata from formal tool analysis: module hierarchies, ports (direction, bit-width), signals, FSM encodings, parameters
<i>properties.json</i>	Generated SVA code with property identifiers, requirement traceability, type classification, line mappings
<i>tracelinks.json</i>	Mappings encoding requirement-to-specification, property-to-requirement, property-to-coverage relationships for bidirectional traceability
<i>formal_results.json</i>	Per-property outcomes with formal status (proven, CEX, vacuous), proof depth, runtime, diagnostic artifacts (VCD)
<i>cex_cases.json</i>	Counterexample tracking with property identifiers, VCD paths, failure line numbers, correction attempt history for iterative debugging
<i>coverage_metrics.json</i>	Aggregated statistics: reachability percentages, proof core ratio, vacuity counts, dead code classifications driving coverage iterations
<i>run_context.json</i>	Execution snapshot with run identifiers, file paths, iteration counts, tool versions, timestamps, configuration
<i>nodes.csv</i>	Graph node export with unique IDs, type labels (spec_chunk, requirement, property, formal_result), run IDs, JSON attributes
<i>edges.csv</i>	Graph edge export with source/destination IDs, relationship types (derives_from, validates, proves, fails, covers), run IDs, JSON attributes

while preserving traceability from requirements to properties and outcomes

In each run, requirements and RTL metadata are extracted into typed JSON IRs, followed by property generation and formal verification. Syntax and CEX correction along with coverage improvement are triggered based on tool outputs and updated IR artifacts as shown in Fig. 1.

Listing 1: IR core entity types and relationships

Entities:	
SpecChunk	- specification fragments
Requirement	- functional requirements
Property	- generated SVA code
FormalResult	- verification outcomes
CoverageMetrics	- RTL coverage tracking
Relationships:	
Requirement	--> SpecChunk (provenance)
Property	--> Requirement (validation)
Property	--> FormalResult (outcomes)
Property/RTL	--> CoverageMetrics (coverage)

A. Intermediate Representations and Knowledge Graphs

Verification artifacts are stored as typed JSON IRs to enable deterministic orchestration and cross-iteration comparison. Table I enumerates the IR artifacts, and Listing 1 summarizes the core entity types and trace links used for requirement-centric, property-centric, and coverage-centric retrieval.

The runtime KG is constructed from *nodes.csv* and *edges.csv* exported from the IR artifacts. The KG acts as an artifact registry with property-granular updates: when a property is regenerated or corrected, only its downstream evidence (for example, formal results, CEXs, and coverage records) is invalidated and recomputed, enabling focused re-verification while preserving end-to-end traceability.

The runtime KG constructs bounded context by traversing from a task anchor node, typically a requirement or a property. For property generation, the neighborhood includes the target requirement, its parent specification fragment, relevant RTL objects (module scope and signals), and related properties validating sibling requirements. For repair tasks, the neighborhood

also includes tool outputs such as compilation diagnostics, CEX traces, and coverage gap reports. RTL metadata is indexed by hierarchical signal identifiers, enabling automatic mapping from specification-level signal mentions to RTL hierarchical paths.

B. Multi-Agent Orchestration

Verification is decomposed into specialized agents to separate concerns that would be difficult for a single monolithic prompt: specification interpretation, RTL signal grounding, temporal logic synthesis, and tool-driven debugging. Each agent operates with a KG-bundled context neighborhood appropriate to its task, emits structured updates to IR objects and property code, and hands off to the next agent through explicit coordination protocols. This enables deterministic routing and complete auditability of the verification workflow. Fig. 1a illustrates the agent interactions during property generation, while Fig. 1b shows the CEX correction and coverage improvement systems.

1) *Property Generation Agents*: Requirements are translated into SVA properties through a multi-agent workflow. The *sva_lead* oversees the property generation process by coordinating agents and ensuring verification strategy alignment with requirements. RTL metadata extracted from the parser populates the *design_model* IR artifact with module hierarchies, ports, signals, and parameters. For each requirement, the KG provides a neighborhood containing the requirements, parent specification fragment, RTL signals, and rulebook guidelines. The *spec_analyst* decomposes requirements into testable conditions: trigger events, responses, timing constraints, and exceptions. The *sva_author* translates these into SVA syntax using assertions, temporal operators, and assume directives. The *sva_reviewer* validates against specification adherence and rulebooks. The *sva_patcher* corrects properties based on *sva_reviewer* feedback. The *code_extractor* assembles approved properties into a SVA file with macros and property declarations.

2) *Syntax Correction Agents*: LLM-generated syntax errors in properties are addressed through a multi-agent correction

TABLE II
Evaluation of the proposed methodology

Evaluation Category	Evaluation Metric	FIFO [14]	Lemming [15]	ALU [15]	CICD [15]	BST [15]	TTC Counter [15]	CSR [15]
Property generation	#Properties (T P F)	26 17 9	51 50 1	2 1 1	15 12 3	34 18 16	41 30 11	42 26 16
	#Vacuous properties	0	0	0	0	7	2	5
	%Reachable RTL coverage	88.2	94.7	95.8	72.6	73.1	59.0	33.8
Syntax correction	#Fix attempts	1	0	3	1	1	1	1
CEX correction	#Properties (C NC)	8 1	0 1	0 1	0 3	0 16	0 11	0 16
Coverage improvement	#Properties (T P F)	26 25 1	51 50 1	2 1 1	15 12 3	37 20 17	41 30 11	42 32 10
	#Vacuous properties	1	0	0	0	7	2	5
	%Reachable RTL coverage	88.2	94.7	95.8	71.8	73.1	59.0	84.6
End-to-end results	%Formal coverage	93.8	97.32	99.41	90.83	86.46	82.98	93.82
Knowledge Graph size	#Nodes #Edges	231 1054	402 2157	116 396	165 688	299 1107	302 1522	338 1397

T - Total, P - Passed, F - Failed, C - Corrected, NC - Not Corrected, FIFO - First In First Out, ALU - Arithmetic Logic Unit, CICD - Cascaded Integrator-Comb Decimator, BST - Binary Search Tree, TTC Counter - Time-to-Completion Counter, CSR (CSR APB Interface Design) - Control and Status Register Advanced Peripheral Bus Interface

strategy. After compilation, the `syntax_analyzer` parses error messages and attributes them to specific properties. The `syntax_fixer` applies deterministic repairs for well-known patterns (undeclared identifiers wrapped with backtick macro syntax or hierarchical paths, undefined macros auto-resolved by adding definitions) and handles complex errors (temporal operators, sequence expressions, type mismatches) using compilation diagnostics, property context, requirement text, and RTL signal declarations. The `syntax_validator` performs isolated testing in a temporary wrapper to prevent cascading errors before reintegration. The `code_extractor` extracts the corrected code. Properties receive maximum three attempts; persistent failures are disabled and logged for manual review.

3) *Counterexample Correction Agents*: Properties failing FV produce CEXs indicating RTL bugs or property issues. VCD waveforms, failure timestamps, and signals are extracted and stored as `FormalResult` IR entities. The `vcd_parser` extracts temporal behavior at failure points. The `spec_assertion_analyzer` classifies root causes: RTL bugs, over-specification, missing assumptions, or under-specification. The `rtl_analyzer` examines RTL source code and traces logic paths; genuine bugs are documented for RTL correction. The `cex_fixer` corrects property issues by adjusting constraints, adding assumptions, or strengthening assertions. The `code_extractor` assembles corrected properties for isolated verification before reintegration. Properties receive maximum three attempts, with persistent failures flagged for manual analysis.

4) *Coverage Improvement Agents*: Syntactically correct properties may still achieve inadequate coverage. Formal tool reports are parsed to extract coverage metrics: unreachable RTL statements, dead code regions and vacuous properties, stored as `CoverageMetrics` IR entities. The `cov_lead_agent` coordinates coverage improvement by prioritizing gaps in functional paths and distinguishing intentionally unreachable defensive code. The `cov_analyzer` performs root cause analysis, examining RTL implementation to identify conditions for unreachable code and querying the KG to determine if existing properties or assumptions prevent those combinations. The `cov_processor` links gaps to requirements through KG traversal, identifying under-verified requirement aspects.

The `cov_improver` generates targeted properties using cover directives for reachability and assertions for correctness. The `code_extractor` assembles coverage properties for iterative formal runs.

IV. EVALUATION

The system was evaluated on seven benchmark designs: FIFO, Lemming, ALU, CIC Decimator, Binary Search Tree (BST), TTC Counter, and CSR APB Interface, using the multi-agent flow with the GPT-5.2 model and Cadence JasperGold for FV, as shown in Table II. Overall, the workflow consistently produces runnable SVA at scale. Syntax correction remains low effort across the completed designs, which indicates that KG-grounded context, especially deterministic signal and hierarchy resolution via the `design_model` IR, reduces common front-end issues such as undeclared identifiers, macro mismatches, and incorrect hierarchical references.

The main differences across designs appear after compilation, during CEX-based refinement and coverage improvement. FIFO and CIC Decimator benefit from iterative refinement, with improved pass rates and higher end-to-end formal coverage after the correction loops. This suggests that many initial failures are caused by local issues such as missing guards, overly strong assumptions, or boundary-condition constraints, which can be corrected using retrieved context and CEX evidence. TTC Counter shows moderate convergence: a non-trivial fraction of initially failing properties become provable after CEX correction, but overall coverage remains lower than the strongest benchmarks, consistent with limited reachable-state coverage in the design. CSR APB Interface shows a different pattern: initial property quality is lower, but the coverage loop yields a clear improvement in passing properties, indicating that additional targeted properties help exercise previously uncovered behavior.

In contrast, Lemming and ALU show limited benefit from automated correction. Most properties pass early, and the remaining failures do not improve significantly, suggesting that the unresolved cases require complex temporal reasoning such as reset behavior, FSM transition intent, or multi-cycle arithmetic meaning. BST remains the most difficult: many properties are generated, but a large fraction remain failing and CEX correction does not converge, which is consistent with the need

for global data-structure invariants and inductive reasoning that exceed local patching strategies.

Vacuous-property behavior further separates the benchmarks. CIC Decimator and BST exhibit more vacuous properties, indicating some checks can pass without strongly constraining behavior under the default environment. This motivates stronger environment modeling or non-vacuity constraints earlier in the loop so coverage improvements reflect exercised behavior rather than trivially satisfied properties.

KG size grows with design complexity and trace link density, but it does not directly predict verification success. Overall, the KG improves reliability and repeatability of artifact management and context retrieval, particularly for property generation and syntax stabilization, while semantic debugging remains the main limiting factor on harder benchmarks.

V. CONCLUSION

This paper presented a KG-centric, multi-agent workflow for LLM-assisted FV that addresses specification-to-RTL grounding through structured IRs and bounded context retrieval. Specifications, RTL metadata, properties, tool outcomes, and coverage evidence are represented as typed JSON artifacts and exported as nodes and edges for constructing a requirement-based runtime KG that supports property generation and post-run refinement loops. Across the evaluated benchmarks, the approach consistently produces compilable, traceable SVAs with low syntax-fix overhead, while the largest remaining challenge is semantic correction for properties that require deeper temporal reasoning or global invariants. Importantly, as LLMs improve, they can better leverage the KG's structured context and traces to improve property generation and downstream formal verification. Future work will focus on improving CEX correction with richer temporal reasoning patterns and inductive invariant generation for stateful designs, and on refining coverage-gap classification to better separate unreachable code from true verification targets.

REFERENCES

- [1] R. Kande et al., "(Security) Assertions by Large Language Models," *IEEE Transactions on Information Forensics and Security*, 2024.
- [2] Z. Yan et al., "AssertLLM: Generating Hardware Verification Assertions from Design Specifications via Multi-LLMs," in *Proceedings of the 30th ASPDAC Conference*, ACM, 2025.
- [3] B. Mali et al., "ChIRAAG: ChatGPT Informed Rapid and Automated Assertion Generation," in *IEEE ISVLSI*, 2024.
- [4] A. Kumar et al., "Saarthi: The First AI Formal Verification Engineer," in *DVCon US*, 2025.
- [5] D. N. Gadde et al., "Hey AI, Generate Me a Hardware Code! Agentic AI-based Hardware Design & Verification," in *38th SBCCI*, 2025.
- [6] Y. Bai et al., "AssertionForge: Enhancing Formal Verification Assertion Generation with Structured Representation of Specifications and RTL," in *IEEE ICLAD*, 2025.
- [7] F. Wu et al., "Spec2Assertion: Automatic Pre-RTL Assertion Generation using LLMs with Progressive Regularization," arXiv, 2025.
- [8] A. Ayalasomayajula et al., "LASP: LLM Assisted Security Property Generation for SoC Verification," in *ACM/IEEE 6th MLCAD*, 2024.
- [9] H. Lyu et al., "AssertMiner: Module-Level Spec Generation and Assertion Mining using Static Analysis Guided LLMs," arXiv, 2025.
- [10] H. A. Qudus et al., "Enhanced VLSI Assertion Generation: Conforming to High-Level Specifications and Reducing LLM Hallucinations with RAG," in *DVCon Europe*, 2024.
- [11] S. Paul et al., "LISA: LLM Informed Systemverilog Assertion generation with RAG and Chain-of-Thought," in *IEEE ISVLSI*, 2025.
- [12] V. Pulavarthi et al., "Are LLMs Ready for Practical Adoption for Assertion Generation?" In *DATE Conference*, 2025.
- [13] K. Maddala et al., "LAAG-RV: LLM Assisted Assertion Generation for RTL Design Verification," in *IEEE 8th ITC India*, 2024.
- [14] *OpenCores*, <https://opencores.org>.
- [15] N. Pinckney et al., *CVDP: A Next-Generation Benchmark Dataset for Evaluating LLMs and Agents on RTL Design and Verification*, 2025.